

VIT BHOPAL UNIVERSITY

[Department of Computer Science (Artificial Intelligence and Machine Learning)]

PROJECT REPORT

CONTACT BOOK MANAGEMENT APPLICATION

A menu-driven console application built in Python

Detail	Information
Submitted by	HARSHRAJ
Registration No.	26BAI10195
Course / Semester	[PYTHON ESSENTIALS, 1 st SEMESTER]
Subject	[INTRODUCTION TO PROBLEM SOLVING AND PROGRAMMING]
Under the guidance of	[Dr. RAJIT NAIR]
Academic year	[2026 - 2027]
Date of submission	[30 / 09 / 2026]

Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams (Use Case, Workflow, Sequence)
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

A contact book is one of the most familiar applications in daily life: everyone keeps names, phone numbers and email addresses somewhere. This project is a simple Contact Book Management Application written in Python. It runs in the terminal and lets the user create, view, update, delete, search and count contacts through a numbered menu.

The project was built as a beginner-level exercise to practise core Python concepts: dictionaries (including nested dictionaries), loops, conditional statements, user input and formatted output. Although small, the program covers all four basic data operations known as CRUD (Create, Read, Update, Delete), which are the foundation of almost every real-world software system.

Scope: a single-user, console-based application that keeps its data in memory while the program is running. The application is contained in one Python file of about 80 lines and needs no external libraries.

3. Problem Statement

People need a quick and organised way to store the details of the people they know. Writing them on paper or in scattered files makes it slow to find a number, easy to create duplicates and hard to correct outdated information.

The problem addressed by this project is to design and implement a small software tool that:

- stores the name, age, email address and mobile number of each contact;
- prevents two contacts from being saved under the same name;
- allows a saved contact to be looked up, corrected or removed at any time;
- let the user find contacts by typing only part of a name; and
- gives clear feedback for every action, including when something goes wrong (for example, a contact is not found).

4. Functional Requirements

Functional requirements describe what the system must do. The table below lists the requirements implemented in this project.

ID	Requirement	Description
FR1	Main menu	The system shall display a menu of seven options and keep showing it after each operation until the user chooses Exit.
FR2	Create contact	The system shall accept a name, age, email and mobile number and save them as a new contact.
FR3	Duplicate check	The system shall refuse to create a contact whose name already exists and shall inform the user.
FR4	View contact	The system shall display the age, mobile number and email of the contact whose name is entered.

ID	Requirement	Description
FR5	Update contact	The system shall replace the age, email and mobile number of an existing contact with newly entered values.
FR6	Delete contact	The system shall remove an existing contact from the contact book.
FR7	Search contact	The system shall list every contact whose name contains the entered text, ignoring upper and lower case.
FR8	Count contacts	The system shall display the total number of contacts currently stored.
FR9	Error messages	The system shall show a clear message when a contact is not found, a search has no result or a menu option is invalid.
FR10	Exit	The system shall end the program with a goodbye message when the user chooses option 7.

5. Non-functional Requirements

Non-functional requirements describe how well the system should work rather than what it does.

Quality	Requirement and how it is met
Usability	The interface is a plain numbered menu. Every prompt says what to type, and every action ends with a readable success or error message in capital letters.
Performance	Contacts are stored in a Python dictionary, so looking up a name is almost instant. Every operation responds immediately for the small data size expected.
Portability	The program uses only standard Python 3 and runs on Windows, Linux or macOS with no extra installation.
Reliability	Duplicate names are rejected, and missing contacts are handled with a message instead of an error, so the menu keeps working.
Maintainability	The code is short, uses meaningful variable names and has a comment above every operation, so a beginner can read and change it easily.
Simplicity	A single file with no dependencies makes the project easy to copy, submit and run.
Data scope	Data is held in memory only, so the contacts last for one run of the program. This was a deliberate scope decision for the first version.

6. System Architecture

The application follows a simple three-layer structure, all inside one script. The user talks to the console, the main loop decides which operation to run, and the operations read or change one dictionary that holds all the data.

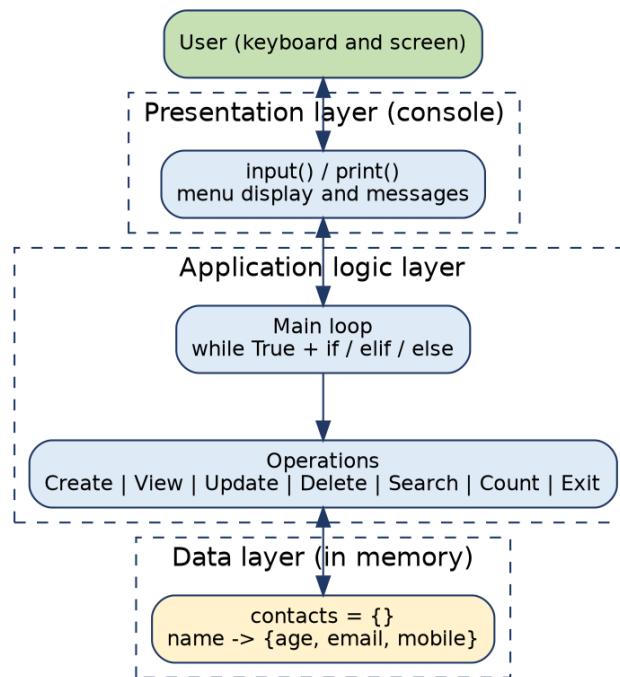


Figure 1: System architecture of the Contact Book application

Presentation layer: the built-in `input()` and `print()` functions show the menu, read the user's choice and display messages.

Application logic layer: a `while True` loop repeatedly shows the menu, and an `if / elif / else` chain sends the choice to the matching operation (Create, View, Update, Delete, Search, Count or Exit).

Data layer: a dictionary named `contacts`. Each key is a contact name, and each value is another dictionary holding the age, email and mobile number.

7. Design Diagrams

7.1 Use Case Diagram

There is one actor, the User, who interacts with the system through seven use cases. There is no login and no administrator because the program is meant for one person on one computer.

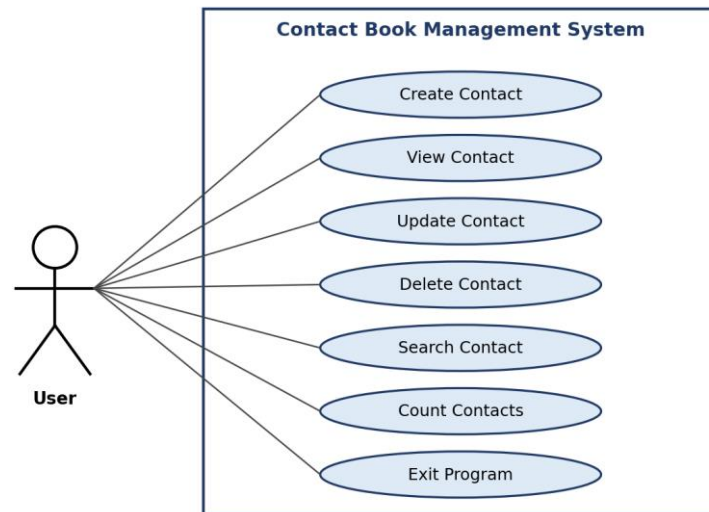


Figure 2: Use case diagram

7.2 Workflow Diagram

The workflow diagram shows how control moves through the program. After the dictionary is created, the menu is shown in an endless loop. The chosen number is compared with each option in turn; only option 7 leaves the loop.

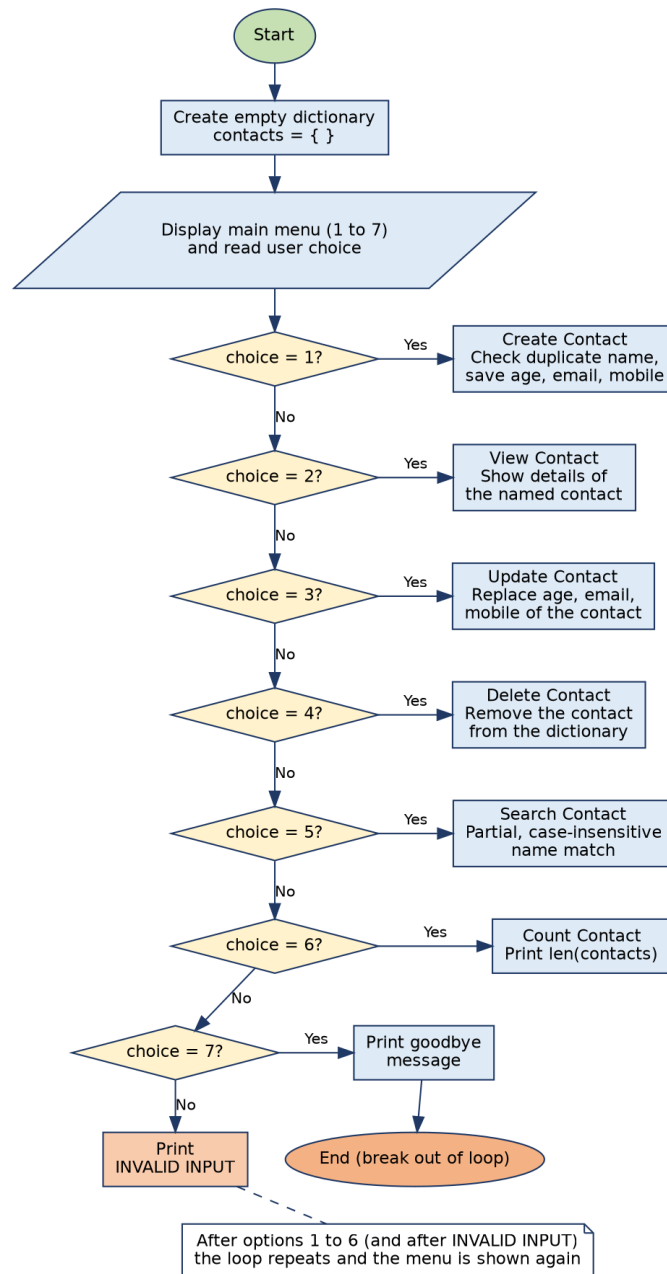


Figure 3: Workflow (flowchart) of the application

7.3 Sequence Diagrams

Two sequence diagrams are given to show how the user, the main menu loop and the contacts dictionary exchange messages. Dashed arrows are replies or prompts shown to the user.

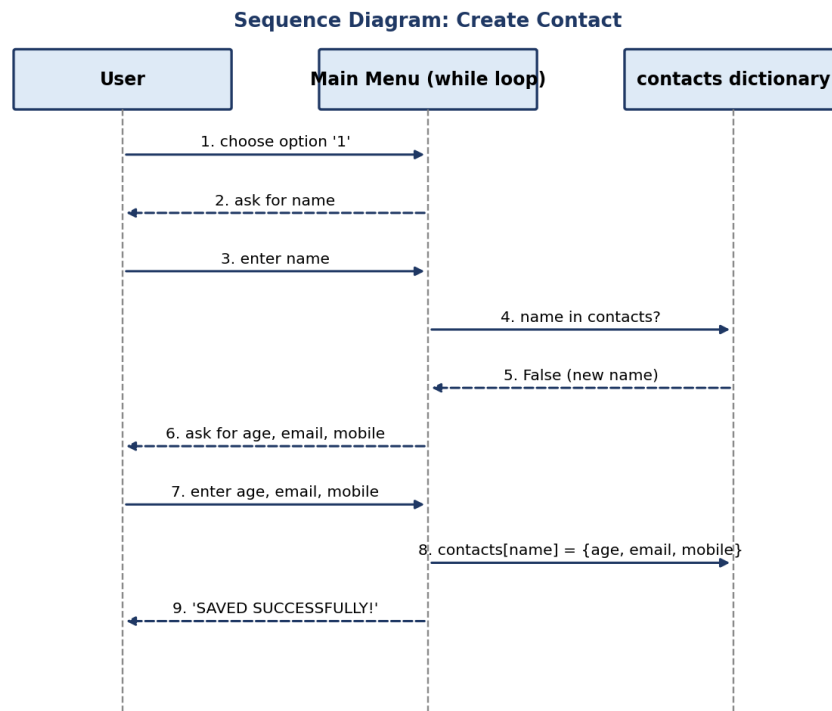


Figure 4: Sequence diagram for creating a contact

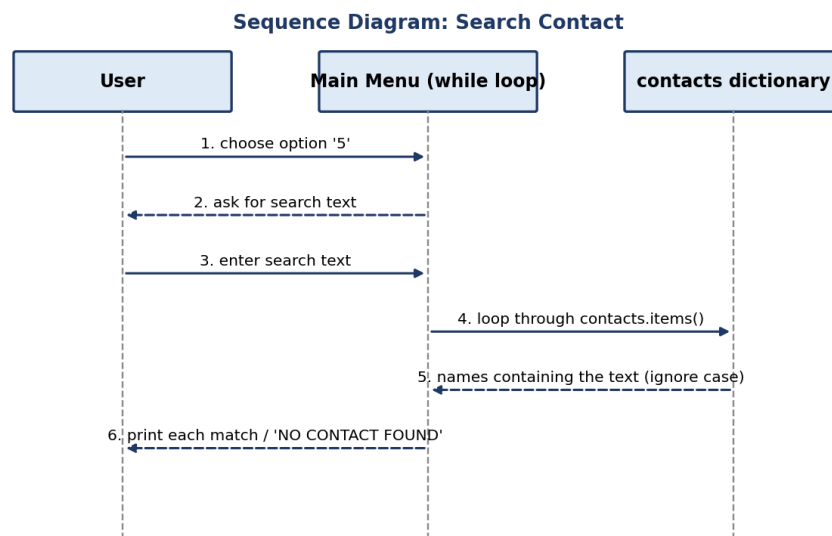


Figure 5: Sequence diagram for searching a contact

8. Design Decisions & Rationale

Decision	Rationale
Use a dictionary keyed by contact name	A dictionary gives fast lookup by name and makes it very easy to check whether a name already exists (name in contacts). This also stops duplicate names automatically.
Nested dictionary for details	Storing age, email and mobile inside a small dictionary gives each piece of data a clear label (for example contact['email']), which is easier to read than remembering list positions.

Decision	Rationale
while True loop with break	A menu program must keep running until the user decides to stop. The loop repeats the menu and break is used only for the Exit option.
if / elif / else chain	There are only seven fixed choices, so a simple chain is the easiest structure for a beginner to write and to follow. The final else handles invalid input.
Case-insensitive partial search	Both the typed text and the stored names are converted with .lower() and matched with the in operator, so 'rah' finds 'Rahul'. Users rarely remember exact spelling or capital letters.
Age stored as an integer	The age is converted with int() so it is stored as a number, while email and mobile stay as text. Mobile numbers are text because they are never used in arithmetic and may start with zero.
In-memory storage	Keeping data in memory avoids file handling and keeps the first version focused on the core logic. Saving to a file is planned as a future enhancement.
Single-file program	One file with no imports is simple to run, share and submit.
Clear capital-letter messages	Success and error messages are written in capitals so they stand out from the prompts.

9. Implementation Details

9.1 Environment

- Language: Python 3 (no external libraries)
- Editor: any text editor or IDE such as VS Code, IDLE or PyCharm
- Run command: `python contact_book.py`

9.2 Data structure

All contacts live in a single dictionary created at the top of the program. A stored contact looks like this:

```
contacts = {
    'Rahul': {'age': 21, 'email': 'rahul@example.com', 'mobile': '9876543210'}
}
```

9.3 Main loop and menu

The program prints the title and the seven options, reads the choice with `input()` and then checks it against each option. The loop only stops when the user enters 7.

```
while True:
    print("\n***WELCOME TO PYTHON CONTACT BOOK MANAGEMENT APP***")
    print("1. Create Contact")
    ...
    choice = input("Please choose an option (1/2/3/4/5/6/7) : ")
```

9.4 Operations

Option	How it is implemented
1. Create	Reads a name. If name in contacts, an “already exists” message is printed. Otherwise age, email and mobile are read and stored with contacts[name] = {...}.
2. View	Reads a name. If it is present, the record is fetched with contact = contacts[name] and its age, mobile and email are printed from that record. Otherwise “CONTACT NOT FOUND!” is printed.
3. Update	Checks that the name exists, reads new age, email and mobile, and overwrites the old record with a new dictionary.
4. Delete	Checks that the name exists and removes it using del contacts[name].
5. Search	Loops through contacts.items(). For every name where the search text (in lower case) appears inside the name (in lower case), the contact’s details are printed. A found flag decides whether the “no contact found” message is needed.
6. Count	Prints len(contacts).
7. Exit	Prints a goodbye message and uses break to leave the loop.
Anything else	Prints “INVALID INPUT” and the menu is shown again.

9.5 Key code snippets

Creating a contact with a duplicate check:

```
name = input("Enter the name you want the contact to be saved = ")
if name in contacts:
    print(f'CONTACT NAME {name} ALREADY EXISTS!')
else:
    age = input("Enter age = ")
    email = input("Enter email = ")
    mobile = input("Enter mobile number = ")
    contacts[name] = {'age':int(age), 'email':email, 'mobile':mobile}
    print(f'CONTACT NAME {name} HAS BEEN SAVED SUCCESSFULLY!')
```

Searching by part of a name:

```
found = False
for name, contact in contacts.items():
    if search_name.lower() in name.lower():
        print(f"Found - Name:{name}, Age:{contact['age']}, Mobile
Number:{contact['mobile']}, Email:{contact['email']}")
        found = True
if not found:
    print("NO CONTACT FOUND WITH THAT NAME")
```

10. Screenshots / Results

The screenshots below were taken from actual runs of the program in a terminal. Text in green is what the user typed.

```
Command Prompt - python contact_book.py

***WELCOME TO PYTHON CONTACT BOOK MANAGEMENT APP***
1. Create Contact
2. View Contact
3. Update Contact
4. Delete Contact
5. Search Contact
6. Count Contact
7. Exit
Please choose an option (1/2/3/4/5/6/7) : 1
```

Figure 6: Main menu of the application

```
Command Prompt - python contact_book.py

Please choose an option (1/2/3/4/5/6/7) : 1
Enter the name you want the contact to be saved = Rahul
Enter age = 21
Enter email = rahul@example.com
Enter mobile number = 9876543210
CONTACT NAME Rahul HAS BEEN SAVED SUCCESSFULLY!
Please choose an option (1/2/3/4/5/6/7) : 1
Enter the name you want the contact to be saved = Rahul
CONTACT NAME Rahul ALREADY EXISTS!
Please choose an option (1/2/3/4/5/6/7) : 7
GOOD BYEE...CLOSING THE PROGRAM
```

Figure 7: Creating a contact and the duplicate-name message

```
Command Prompt - python contact_book.py

Please choose an option (1/2/3/4/5/6/7) : 1
Enter the name you want the contact to be saved = Rahul
Enter age = 21
Enter email = rahul@example.com
Enter mobile number = 9876543210
CONTACT NAME Rahul HAS BEEN SAVED SUCCESSFULLY!
Please choose an option (1/2/3/4/5/6/7) : 2
Enter Contact name to view = Rahul
Name: Rahul, Age:21, Mobile Number:9876543210, Email:rahul@example.com
Please choose an option (1/2/3/4/5/6/7) : 2
Enter Contact name to view = Amit
CONTACT NOT FOUND!
Please choose an option (1/2/3/4/5/6/7) : 7
GOOD BYEE...CLOSING THE PROGRAM
```

Figure 8: Viewing an existing contact and a contact that does not exist

```
Command Prompt - python contact_book.py

Please choose an option (1/2/3/4/5/6/7) : 1
Enter the name you want the contact to be saved = Rahul
Enter age = 21
Enter email = rahul@example.com
Enter mobile number = 9876543210
CONTACT NAME Rahul HAS BEEN SAVED SUCCESSFULLY!
Please choose an option (1/2/3/4/5/6/7) : 3
Enter Contact name that you want to update = Rahul
Enter updated age = 22
Enter updated email = rahul.new@example.com
Enter updated mobile number = 9000000001
CONTACT NAME Rahul HAS BEEN UPDATED SUCCESSFULLY!
Please choose an option (1/2/3/4/5/6/7) : 2
Enter Contact name to view = Rahul
Name: Rahul, Age:22, Mobile Number:9000000001, Email:rahul.new@example.com
Please choose an option (1/2/3/4/5/6/7) : 7
GOOD BYEE...CLOSING THE PROGRAM
```

Figure 9: Updating a contact and viewing the new details

```
Command Prompt - python contact_book.py

Please choose an option (1/2/3/4/5/6/7) : 1
Enter the name you want the contact to be saved = Rahul
Enter age = 21
Enter email = rahul@example.com
Enter mobile number = 9876543210
CONTACT NAME Rahul HAS BEEN SAVED SUCCESSFULLY!
Please choose an option (1/2/3/4/5/6/7) : 1
Enter the name you want the contact to be saved = Priya
Enter age = 22
Enter email = priya@example.com
Enter mobile number = 9123456780
CONTACT NAME Priya HAS BEEN SAVED SUCCESSFULLY!
Please choose an option (1/2/3/4/5/6/7) : 5
Enter contact name to search = rah
Found - Name:Rahul, Age:21, Mobile Number:9876543210, Email:rahul@example.com
Please choose an option (1/2/3/4/5/6/7) : 5
Enter contact name to search = xyz
NO CONTACT FOUND WITH THAT NAME
Please choose an option (1/2/3/4/5/6/7) : 6
Total contacts in your contact book : 2
Please choose an option (1/2/3/4/5/6/7) : 7
GOOD BYEE...CLOSING THE PROGRAM
```

Figure 10: Searching by part of a name, a search with no result, and counting contacts

```
Command Prompt - python contact_book.py

Please choose an option (1/2/3/4/5/6/7) : 1
Enter the name you want the contact to be saved = Rahul
Enter age = 21
Enter email = rahul@example.com
Enter mobile number = 9876543210
CONTACT NAME Rahul HAS BEEN SAVED SUCCESSFULLY!
Please choose an option (1/2/3/4/5/6/7) : 4
Enter contact name to delete = Rahul
CONTACT NAME Rahul HAS BEEN DELETED SUCCESSFULLY!
Please choose an option (1/2/3/4/5/6/7) : 6
Total contacts in your contact book : 0
Please choose an option (1/2/3/4/5/6/7) : 4
Enter contact name to delete = Rahul
CONTACT NOT FOUND!
Please choose an option (1/2/3/4/5/6/7) : 7
GOOD BYEE...CLOSING THE PROGRAM
```

Figure 11: Deleting a contact, counting, and deleting a contact that is not present

```
Command Prompt - python contact_book.py

Please choose an option (1/2/3/4/5/6/7) : 9
INVALID INPUT
Please choose an option (1/2/3/4/5/6/7) : abc
INVALID INPUT
Please choose an option (1/2/3/4/5/6/7) : 7
GOOD BYEE...CLOSING THE PROGRAM
```

Figure 12: Invalid menu input and exiting the program

11. Testing Approach

The application was tested manually using black-box testing: each menu option was run with normal input and with input that should cause an error message, and the output was compared with the expected result. Every option was tested in the same terminal session so that the effect of one operation on the next (for example, delete followed by count) could also be checked.

Test cases and results:

No.	Feature	Input	Expected result	Status
T1	Create	Option 1, name Rahul, age 21, valid email and mobile	Contact saved; success message shown	Pass
T2	Create (duplicate)	Option 1, name Rahul again	"ALREADY EXISTS" message; nothing is overwritten	Pass
T3	View	Option 2, name Rahul	Age, mobile and email of Rahul are shown	Pass
T4	View (missing)	Option 2, name Amit	"CONTACT NOT FOUND!"	Pass

No.	Feature	Input	Expected result	Status
T5	Update	Option 3, name Rahul, new age, email, mobile	Update message; View shows the new values	Pass
T6	Update (missing)	Option 3, name Ghost	"CONTACT NOT FOUND!"	Pass
T7	Search	Option 5, text "rah" with Rahul and Priya saved	Only Rahul is listed, with the correct details	Pass
T8	Search (no match)	Option 5, text "xyz"	"NO CONTACT FOUND WITH THAT NAME"	Pass
T9	Count	Option 6 after adding two contacts	Total contacts: 2	Pass
T10	Delete	Option 4, name Rahul	Delete message; count drops by one	Pass
T11	Delete (missing)	Option 4, name Rahul again	"CONTACT NOT FOUND!"	Pass
T12	Invalid menu input	Options 9 and abc	"INVALID INPUT"; menu shown again	Pass
T13	Exit	Option 7	Goodbye message; program ends	Pass

All thirteen test cases produced the expected output. Testing also showed that the age field must be a number, which is discussed in Sections 12 and 14.

12. Challenges Faced

- **Understanding nested dictionaries.** It took some practice to see that `contacts[name]` returns another dictionary and that the details must then be read with keys such as `['age']`.
- **Wrong details shown in View and Search.** During testing, the first version printed the variables `age`, `mobile` and `email` that were typed most recently instead of reading them from the stored contact, so the wrong person's details could appear. The fix was to read every value from the contact record itself, for example `contact['age']`. This taught the importance of testing with more than one contact.
- **Preventing duplicate names.** Using the `in` operator on the dictionary before saving solved this in one line.
- **Case-insensitive partial search.** Comparing the exact text failed for capital letters, so both the search text and the name are converted to lower case first.
- **Controlling the loop.** Deciding where to use `break` so that only the Exit option ends the program, while all other options return to the menu.
- **Input handling.** `input()` always returns text, so the age has to be converted with `int()`. If a non-numeric age is typed, Python raises a `ValueError`. Checking input properly is listed as an enhancement.

13. Learnings & Key Takeaways

- How to design and use dictionaries and nested dictionaries to store structured data.
- How to build a menu-driven program with a while loop and an if / elif / else chain.
- How to implement the four CRUD operations, plus search and count, in a real mini-application.
- How to use f-strings, string methods such as lower(), the in operator and len().
- Always read values from the data structure instead of relying on variables from earlier steps.
- Testing every option, with both valid and invalid input, finds problems that are easy to miss when only checking the normal path.
- Planning with requirements, workflow and sequence diagrams before and after coding makes the program easier to explain and to improve.

14. Future Enhancements

- **Permanent storage:** save contacts to a JSON or CSV file (or an SQLite database) and load them when the program starts, so data is not lost when the program closes.
- **Input validation:** check that age is a number, that the email contains @ and a domain, and that the mobile number has the correct number of digits, using try / except and simple checks.
- **View all contacts:** add an option to list every contact, sorted by name.
- **Case-insensitive lookups:** make View, Update and Delete ignore capital letters in names.
- **Modular code:** move each operation into its own function to make the program cleaner and easier to test.
- **Extra fields:** address, birthday, group and multiple phone numbers per contact.
- **Import and export:** allow contacts to be imported from or exported to a CSV file.
- **Graphical interface:** build a Tkinter window or a web version so that the application is easier for non-technical users.

15. References

- [1] Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
- [2] Python Software Foundation. The Python Tutorial: Data Structures (Dictionaries). <https://docs.python.org/3/tutorial/datastructures.html>
- [3] Python Software Foundation. The Python Tutorial: More Control Flow Tools. <https://docs.python.org/3/tutorial/controlflow.html>
- [4] Python Software Foundation. Built-in Functions (input, int, len, print). <https://docs.python.org/3/library/functions.html>
- [5] Python Software Foundation. The Python Tutorial: Input and Output (Formatted String Literals). <https://docs.python.org/3/tutorial/inputoutput.html>

- [6] Sommerville, I. Software Engineering, 10th ed. Pearson, 2015.
- [7] Object Management Group. Unified Modeling Language (UML) Specification.
<https://www.omg.org/spec/UML/>
- [8] Graphviz Documentation (used to draw the workflow and architecture diagrams).
<https://graphviz.org/documentation/>
- [9] Matplotlib Documentation (used to draw the use case and sequence diagrams).
<https://matplotlib.org/stable/>